

NVRAM TNG

Belkin NVRAM library and command line utility

In broad terms the goals of nvram and libnvram library rewrite were to:

1. Eliminate NVRAM corruption caused by wemoApp crashes.
2. Minimize flash writes to reduce flash wear and to minimize the number of times that we are vulnerable to flash corruption caused by power loss while writing to flash.
3. Minimize heap fragmentation by avoiding the use of malloc as much as possible.
4. Improve overall performance.
5. Improve debugging support.

To achieve these goals the following new features were added to *libbelkin_nvram*.

New features:

1. Flash writes handled by daemon process.
2. Improved handling of NVRAM access semaphore.
3. NVRAM data caching.
4. Delayed writes.
5. NVRAM variable look up optimization.
6. Configurable levels of NVRAM access logging for debugging.
7. Maintenance of NVRAM access statistics for debugging and unit testing.

Flash writes handled by daemon process

One source of NVRAM corruption that has been identified is application crashes caused by threads unrelated to the NVRAM library code that occur while NVRAM is being updated. By moving responsibility for writing flash to a daemon process this vulnerability is eliminated. The daemon process will also facilitate delayed writes to flash further reduce the chances of flash corruption.

The NVRAM daemon *nvramd* is started by the library code when needed, there is no need to start it separately as part of system startup. The daemon code is contained in the *nvram* command line utility as one of it's aliases.

NVRAM access semaphore

Since the previous *libbelkin_nvram* implementation has exhibited failures attributed to improper handling of the NVRAM access semaphore thought was given to improving the robustness semaphore handling. Specifically the previous code assumed that if the semaphore existed when wemoApp started that it was an error and the semaphore was unceremoniously deleted and recreated. A new more conservative approach has been implemented which never deletes an existing semaphore.

In order to recover from the situation where the semaphore was owned by a thread that has crashed or been killed enhanced procedures for taking and releasing semaphores were implemented. When a caller obtains the shared NVRAM semaphore the owner's thread id will be stored in shared memory. Users wishing to take the semaphore use the `sem_timedwait()` API call rather than `sem_wait()` to avoid blocking forever. If `sem_timedwait()` times out the caller checks to see if the semaphore's owner is still running. If the owner is no longer running the caller releases the semaphore on behalf of the previous owner, otherwise the timeout event is logged along with the thread ID of the owner to facilitate debugging.

NVRAM data caching

The first time the NVRAM library is initialized a POSIX shared memory region is created for NVRAM data caching and storage of *libbelkin_nvram* variables. This memory region is shared across all *libbelkin_nvram* users including multiple processes. Since the shared memory region has kernel persistence it continues to exist independently of any *libbelkin_nvram* user ensuring data will not be lost if a *libbelkin_nvram* user crashes. This includes crashes of *nvr amd*.

Access to the share memory region is controlled by a POSIX semaphore within the shared memory itself. At run time the shared memory region is also present as a file with a path of `/dev/shm/nvram`. This is convenient for debugging but it should not be accessed programmatically.

Delayed flash writes

To minimize the number of flash writes needed for a common use case where multiple NVRAM variables are modified at the same time the new library delays writing the NVRAM cache to flash. When a new NVRAM variable is added, an existing NVRAM variable is deleted, or when the value of an NVRAM variable is changed a flash write timer will be started if it is not running already. If the NVRAM daemon is not running it will be started, otherwise it will be signaled that a flash write is pending. When the write timer times out the NVRAM daemon will obtain the NVRAM semaphore, create a copy of the NVRAM variables sorted in order, release the NVRAM semaphore, update the CRC in the copy and then write the copy to flash. This allows NVRAM accesses to the cache to continue in parallel with the time consuming flash erase and write cycles.

To prevent needless writes the NVRAM set routine compares the “new” value with the “old” value and only initiates a write if the value has actually changed.

NVRAM lookup optimization

To speed up access to NVRAM variables for both gets and sets a binary tree algorithm is used to optimize variable lookups. When the NVRAM library is initialized an alphanumerically sorted index is created for look ups. This table is used when NVRAM is updated (committed) so the variables are written to flash in alphanumerical order for the next boot.

Configurable levels of NVRAM access logging for debugging

The new library may be configured to log NVRAM read and/or write access to syslog facilitate debugging. Errors will be logged to syslog as well as sent to stderr when used from the command line *nvr amd* utility. The *nvr amd* utility's `-v` command line option is used to select the desired logging levels.

The **nvr** utilities **-v?** command line switch can be used to list the available logging levels.

```
root@OpenWrt:~# nvr -v?
Verbose logging levels available:
00000001 - Verbose messages
00000002 - Reads
00000004 - Writes
00000008 - Api calls
00000010 - Nvr Daemon related
00000020 - Semaphore accesses
00000040 - Variable lookup related
root@OpenWrt:~#
```

For example to log NVRAM variable and writes a value of 6 would be used (0x2 + 0x4).

Maintenance of NVRAM access statistics for debugging and unit testing

Statistics are kept on the number of NVRAM variable sets, gets, sets that are ignored because the value hadn't changed, number of failed gets, number of times the cache is written to flash, number of errors accessing flash, the reason the last time nvr was reinitialized, etc. The command line utilities **status** command displays the statistics.

```
root@OpenWrt:~# nvr status
Flags: Gemtek
NVRAM Semaphore is available.
NVRAM daemon is not running.

Variables: 42
Sets: 0
IgnoredSets: 6
UnSets: 0
Gets: 46
NotFound: 74
Commits: 0
TotalCommits: 25
Errors: 0
DaemonStarts: 0
DaemonExits: 0
InternalErrs: 0

Last NVRAM Reset:
Cause: Nvr utility restore command
Time: 5/20/2015 8:14:50 (local)

MTD Device: /dev/mtd6
Erase size: 0x10000 (65536)
NVRAM size: 0x10000 (65536)
root@OpenWrt:~#
```

Usage

The preexisting NVRAM access routines defined in Belkin_api.h of GetBelkinParameter(), SetBelkinParameter(), and UnSetBelkinParameter() are still supported however new code is encouraged to use the NVRAM libraries API defined in libnvram.h directly.

libnvram.h:

```
// Set <variable> to string value <value>
// bCommit: 0 - Don't schedule a commit
// bCommit: 1 - Schedule a commit
int NvramSet(char *Variable, char *Value, int bCommit);

// Set <variable> to integer value <value>
// bCommit: 0 - Don't schedule a commit
// bCommit: 1 - Schedule a commit
//
// Note: values are always stored in NVRAM as strings, this is
// simply
// a convenience function to perform the conversion for the
// caller.
int NvramSetInt(char *Variable, int Value, int bCommit);

// Get pointer to value of <Variable> in NVRAM. If <Variable>
// doesn't
// exist a pointer to an empty string is returned.
// NB: Variable
char *NvramGet(char *Variable);

// Get integer value from NVRAM, store it at pInt
// Return 0 value found and converted successfully
int NvramGetInt(char *Variable, int *pInt);

// Remove <Variable> value from NVRAM
// bCommit: 0 - Don't schedule a commit
// bCommit: 1 - Schedule a commit
int NvramUnset(char *Variable, int bCommit);

// Initialize lib_nvram_bd
int NvramInit(int bIgnoreCRC, const char *SyslogName);

// defines for bit NVRAM logging mask
#define LOG_V 0x1
#define LOG_READ 0x2
#define LOG_WRITE 0x4
#define LOG_API_CALLS 0x8
#define LOG_NVRAM_DAEMON 0x10
#define LOG_SEMAPHORE 0x20
#define LOG_FIND 0x40

// Set NVRAM subsystem logging level.
```

```

// bPersist: 0 - Don't save level in NVRAM
// bPersist: 1 - Save level in NVRAM variable 'NvramLogMask'
// The NVRAM subsystem logging level is restored from
'NvramLogMask' if it
// exists on library initialization.
int NvramSetLogMask(int Mask, int bPersist);

// Return integer number of seconds since Unix epoc, or 0 if the
// system time hasn't been set yet
int NvramGetTimeStamp(void);

#ifdef __MIPSEL__
void restoreFactory(void);
#endif

```

Usage help for the ***nvram*** utility may be displayed by running it without any arguments, by using the ***help*** command, or using the -h command line switch.

```

root@OpenWrt:~# nvram help
nvram compiled May  7 2015 11:15:01
usage: nvram [options] <command> [arguments]
Command:
  help      - show nvram usage help
  commit    - Write NVRAM cache to flash
  show      - dump all NVRAM variables and values
  get <variable>
  set <variable>=<value>
  unset <variable>
  restore   - erase NVRAM preserving select values
  reset     - erase all NVRAM contents
  factory   - Reset pseudo EEPROM (i.e. MAC addresses)
  status    - Display NVRAM runtime status and statistics
  shutdown  - stop NVRAM daemon, delete shared memory
  firstread - first read without crc check
  updatecrc - update crc value for openwrt
  write <file> - write file to flash
Options:
  -h          - Show nvram usage help
  -n          - Don't commit (set and unset commands)
  -v<hex>     - Set verbose output mask
  -V<hex>     - Set and save verbose output mask
  -v?         - Display verbose output mask levels
root@OpenWrt:~#

```

Usage help for the ***nvramd*** daemon may be displayed by using the ***help*** command, or using the -h command line switch.

```

root@OpenWrt:~# nvramd -h
nvramd compiled May  7 2015 11:15:01
usage: nvramd [options]
  -h          - Show nvram usage help
  -f          - run in foreground
  -v<hex>     - Set verbose output mask
  -V<hex>     - Set and save verbose output mask
  -v?         - Display verbose output mask levels
root@OpenWrt:~#

```